

小知识点、

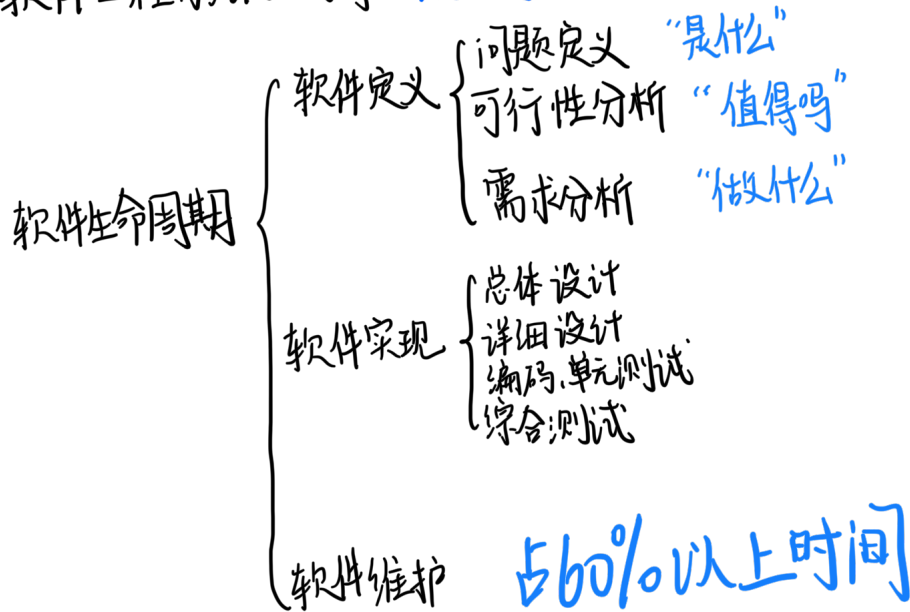
软件危机发生在 **开发与维护的过程中**

原因与软件本身特点、软件开发维护方法不当有关

软件 = 程序 + 数据 + 相关文档

是个工程项目

软件工程方法学三要素：**方法、过程、工具**



瀑布模型 —— “文档驱动”、无法逆转、后期才能知道。

快速原型模型 —— 适应变化

增量模型 —— 灵活 每阶段一个产品

螺旋模型 —— 风险驱动，仅适合大型软件

喷泉模型 —— 边界很浅，“面向对象”、“迭代”、“无间隙”

耦合

内聚

内容 ~

共用 ~

控制 ~

印记(特征) ~

数据 ~

高

低

1. 偶然性 ~

2. 逻辑性 ~

3. 时间性 ~

4. 过程性 ~

5. 通信性 ~

6. 顺序 ~

7. 功能性 ~

8. 信息性 ~

追求低耦合、高内聚

测试是在投入生产前,尽可能多地找出错误

调试由程序员完成,诊断并改正错误.

集成测试

模块测试: 编码与详细设计

子系统测试: 模块接口

系统测试: 设计中

验收测试: 系统需求说明书

α测试: 用户在开发场所进行测试

β测试: 多用户在实际场景下测试

主要设计错误发现迟

非渐增式集成：先进行单元测试，再进行集成测试

渐增式集成：放一块，每次加一些。（要存根程序、驱动程序）

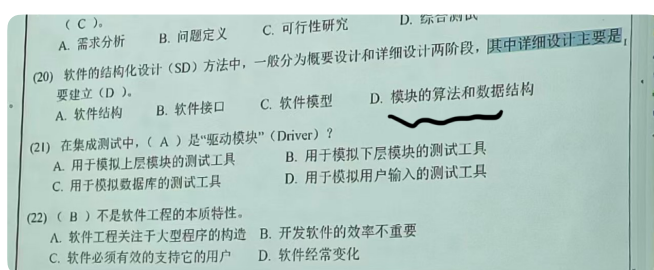
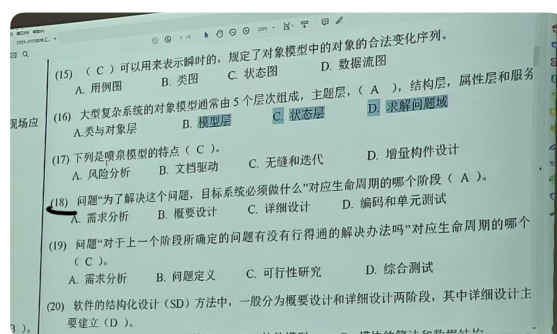
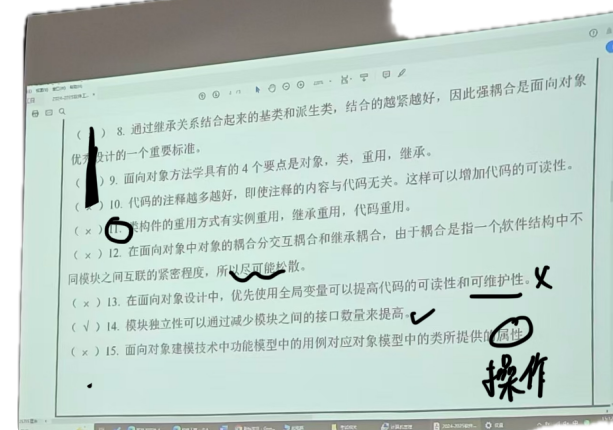
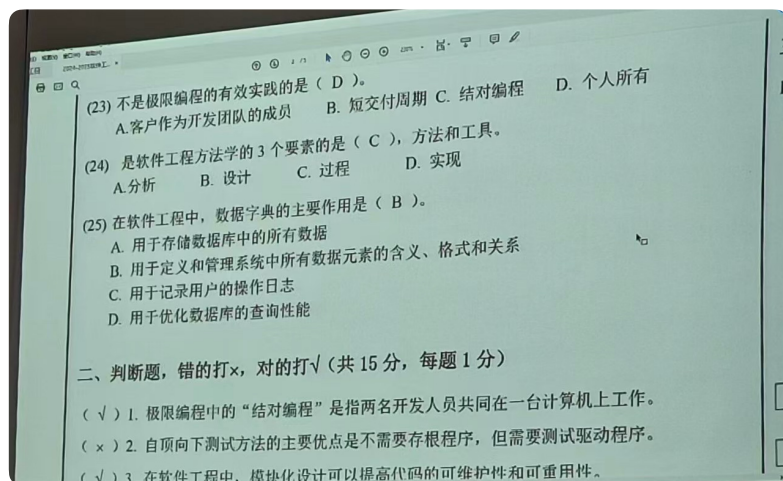
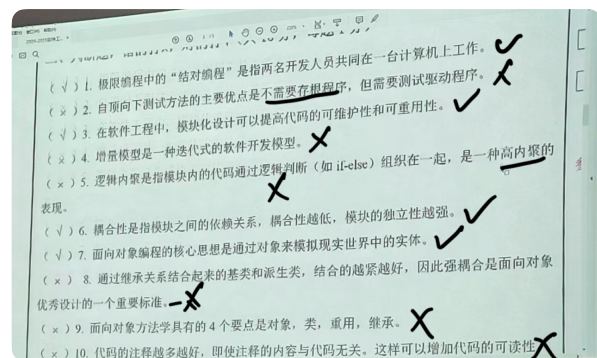
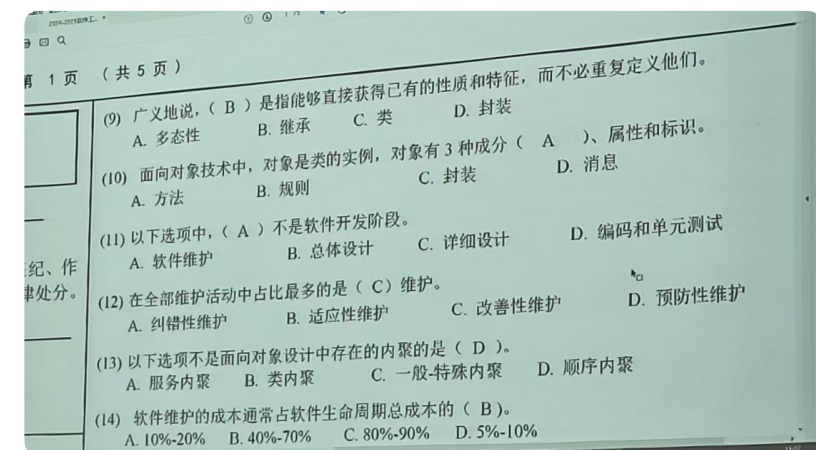
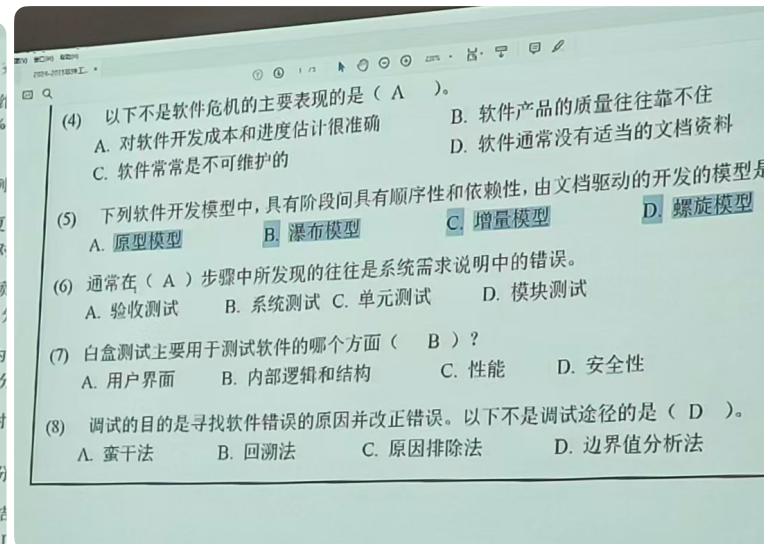
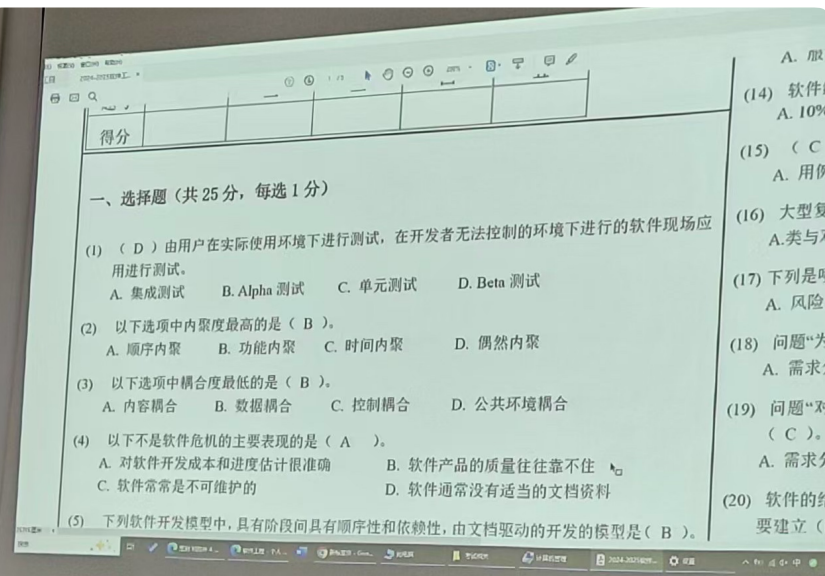
- 自顶向下 ~ 很早，不充分，错误隔离
- 自底往上 ~ 主要设计错误发现迟，但可复用模块充分测试
- 三明治式 ~ 三个优点✓

白盒测试（以逻辑结构为基础）

黑盒测试（划分等价类、边界值分析）

维护中，改善性维护占比最长，50%~66%，预防性维护最少

OO四要点：类、对象、继承、消息。



生命周期模型	优点	缺点	适用范围
瀑布模型	文档驱动的有序方法 简单、易用、直观	交付产品可能不符合客户的要求 不允许变更或者限制变更	在开发中，向下、渐进的路径占据支配地位，即要求在项目开始前，项目的需求已经被很好理解，也很明确；而且项目经理很熟悉为实现这一模型所需要的过程，同时解决方案在项目开始前也很明确。类似的项目如公司的财务系统，库存管理系统及短期项目。一个新的项目不适合瀑布模型，除非处于项目的后期。
快速原型模型	确保交付的产品符合客户的要求	还没有证明无懈可击	当项目开始前项目的需求不明确，或者需要减少项目的不确定性的时候，可以采用原型方法。类似的项目如：需要明确系统的界面，验证一些技术的可行性等。
增量模型	增大投的早期回报	要求开放的结构，可能退化为建造-修补模型	项目开始时明确了大部分的需求，但是需求可能会发生变化的项目；对于市场和用户把握不是很准，需要逐步了解的项目；对于有庞大和复杂功能的系统进行功能改进时需要一步一步实施的项目。
螺旋模型	结合上述所有模型的特性	只能用于大型的内部软件产品，开发者必须精通风险分析和风险排除	风险是主要的制约因素的项目；不确定因素和风险限制了项目进度的项目；用户对自己的需求不是很明确的项目；需要对一些基本的概念进行验证的项目；可能发生一些重大变更的项目；规模很大的项目；采用了新技术的项目。
极限编程	增大投资的早期回报，客户的需求模糊时能编程地工作	还没有被广泛使用	